



TP1 Docker – Student Guide

Everything that was asked, everything that was done,
and how every file works

Group

Oussema BOUCHAALA
Mouin EL DABBABI
Mohamed Amin SADDOUD
Mohamed Helmi LAKHDHAR

This guide is a companion to the official lab report. The report *proves* the work (screenshots, analysis); this guide *explains* it: what Docker concepts are involved, what each file in the folder TP1_docker does, and how to run everything again yourself, step by step.

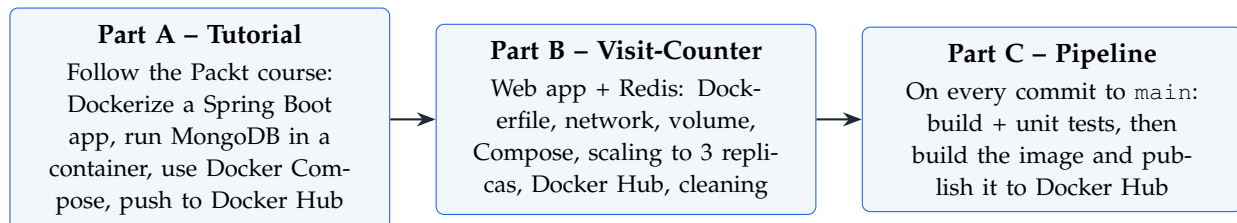
Contents

1	The TP in one page	2
1.1	What was asked	2
1.2	Checklist: requirement → where it is done	2
1.3	Important links	2
2	Docker in 5 minutes (the concepts you need)	3
2.1	Image, container, registry	3
2.2	Network, volume, port mapping	3
2.3	Docker Compose	3
3	The folder TP1_docker file by file	4
4	Part B explained (the main lab)	4
4.1	Architecture	4
4.2	app/app.py – the web service	4
4.3	Dockerfile – line by line	5
4.4	Manual deployment (Part 2)	6
4.5	docker-compose.yml and .env (Part 3)	7
4.6	Scaling: why 3 replicas cannot share port 80	8
4.7	Publication and cleaning (Part 4)	9
4.8	tests/test_app.py – unit tests	9
5	Part A explained (the tutorial)	10
6	Part C explained (the CI/CD pipeline)	11
7	How to run everything again (cheat sheet)	13
8	Glossary	14

1 The TP in one page

1.1 What was asked

The TP has three parts. Part A is the graded video tutorial; Part B is the main lab (an application called *Visit-Counter*); Part C automates everything with a pipeline.



1.2 Checklist: requirement → where it is done

Requirement (TP statement)	Where / how	
Part 0: docker version, docker login, work folder	Docker Desktop 4.43.2; signed in as oussemabouchaala; folder TP_Docker_Bouchaala_...	✓
P1: web app using Redis db-service, key hits, message with container ID	partB-visit-counter/app/app.py (Flask)	✓
P1: Alpine/Slim base, non-root user, cache-friendly Dockerfile	partB-visit-counter/Dockerfile (python:3.12-alpine, USER app)	✓
P1: docker build -t counter-app:1.0 . + size	96.3 MB local, 22.4 MB on Docker Hub	✓
P2: network app-network, volume redis-data, manual docker run, port 80	commands in Section 4.4	✓
P2: robustness test (delete Redis, counter kept)	counter 8 → 9 after docker rm -f	✓
P3: docker-compose.yml + .env, -scale web=3, analysis + load balancer	docker-compose.yml, .env, bonus nginx/nginx.conf	✓
P4: tag + push to Docker Hub; one-command deep clean	oussemabouchaala/counter-app; docker compose down -rmi all -volumes	✓
Report: commented Dockerfile, Compose, screenshots, RUN vs CMD, Hub link	TP_Docker_Report_...pdf (20 pages)	✓
Part C: pipeline on each commit to main: tests, image build, push	.github/workflows/ci-cd.yml (GitHub Actions), runs #2 and #3 green	✓
Part A: tutorial reproduced	partA-tutorial/ (2 Spring Boot apps + MongoDB)	✓

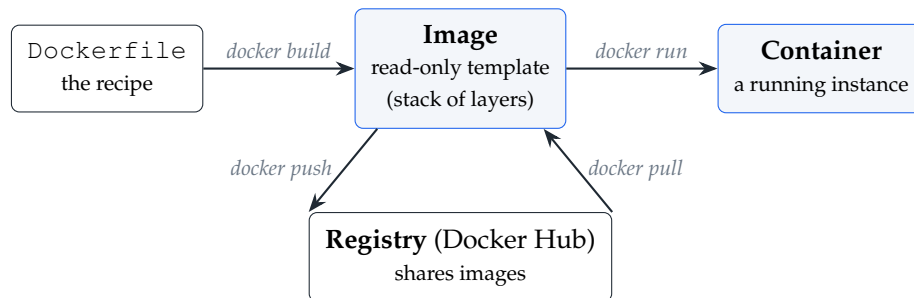
1.3 Important links

Online resources

GitHub repository:	https://github.com/OussemaBouchaala/tp-docker-visit-counter
Pipeline runs:	https://github.com/OussemaBouchaala/tp-docker-visit-counter/actions
Docker Hub image:	https://hub.docker.com/r/oussemabouchaala/counter-app

2 Docker in 5 minutes (the concepts you need)

2.1 Image, container, registry



Image

A packaged application: code + runtime (Python, Java) + libraries + configuration. It is made of *layers*: one per instruction of the Dockerfile. Layers are **cached**: if nothing changed, Docker reuses them (fast builds).

Container

A running image. Each container has its own processes, file system, network address and host-name. Many containers can start from the same image (our 3 replicas).

Registry

An image store. `docker push` uploads, `docker pull` downloads. Docker Hub is the public one.

2.2 Network, volume, port mapping

- **User-defined network**: containers on the same network can talk to each other *by name* (Docker includes a small DNS server). That is why our app reaches Redis at the address `db-service`.
- **Volume**: a storage area managed by Docker that *survives* the container. A container's own files disappear with `docker rm`; the data in a volume stays.
- **Port mapping** `-p HOST:CONTAINER`: makes a container port reachable from your PC. `-p 80:5000` means "http://localhost:80 on Windows goes to port 5000 inside the container".

2.3 Docker Compose

Typing many `docker run` commands is error-prone. **Compose** describes the whole application (services, networks, volumes, variables) in one YAML file, then `docker compose up -d` starts everything and `docker compose down` removes it.

3 The folder TP1_docker file by file

Folder tree

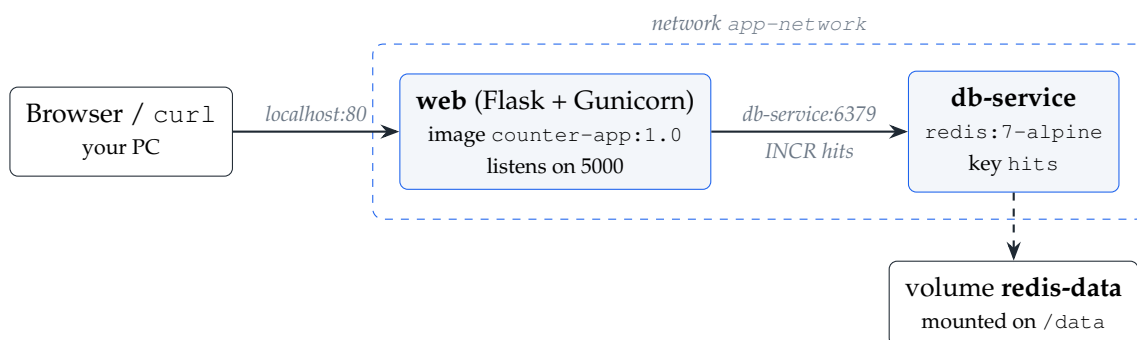
```

TP1_docker/
|-- TP_Docker_Report_Bouchaala_ElDabbabi_Saddoud_Lakhdhar.pdf  <- official report (20 p.)
|-- TP1_Docker_Student_Guide.pdf                               <- this guide
'-- TP_Docker_Bouchaala_ElDabbabi_Saddoud_Lakhdhar/           <- Git repository (on GitHub)
    |-- README.md                                              project summary + pipeline badge
    |-- .gitignore                                              files Git must ignore (caches, LaTeX temp files...)
    |-- .github/workflows/ci-cd.yml                             PART C - the GitHub Actions pipeline
    |-- partA-tutorial/                                         PART A - projects from the video course
        |-- application-one/                                    Spring Boot "welcome" REST API
            |-- Dockerfile                                     multi-stage build (Maven -> JRE)
            |-- .dockerignore, pom.xml, mvnw, .mvn/           Maven build files
            |-- src/...                                         Java code (GreetingsController.java)
            |-- notes.txt                                       commands from the course
            |-- item-service-mongodb/                          Spring Boot CRUD API + MongoDB
                |-- Dockerfile                                 same multi-stage pattern
                |-- docker-compose.yml                         MongoDB + the API together
                |-- src/...                                     controllers, models, repositories
            |-- logs/                                           real build logs (ignored by Git)
    |-- partB-visit-counter/                                    PART B - the Visit-Counter application
        |-- app/app.py                                          the Flask web service
        |-- app/requirements.txt                                runtime libraries (Flask, redis, gunicorn)
        |-- requirements-dev.txt                                + test libraries (pytest, fakeredis)
        |-- tests/test_app.py                                   4 unit tests (used by the pipeline)
        |-- Dockerfile                                          the image of the web service
        |-- .dockerignore                                       files NOT sent to the Docker build
        |-- docker-compose.yml                                  web + db-service (+ lb) orchestration
        |-- .env                                                 variables used by the Compose file
        |-- nginx/nginx.conf                                     bonus load balancer configuration
    '-- report/                                                 LaTeX source of the report
        |-- main.tex, main.pdf
        |-- screenshots/                                        all the real screenshots
        '-- code/                                                copies of the files included in the report

```

4 Part B explained (the main lab)

4.1 Architecture



Each visit: the browser calls the web service → the service sends `INCR hits` to Redis (atomic +1) → Redis returns the new value → the page shows *"Bonjour ! Cette page a été vue X fois. Je suis le conteneur [ID]"*. Redis writes every change to its append-only file in the volume, so the number survives.

4.2 app/app.py – the web service

```

import os
import socket
import time

import redis
from flask import Flask

app = Flask(__name__)

REDIS_HOST = os.getenv("REDIS_HOST", "db-service")
REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))

# The client is lazy: no connection is opened until the first command.
cache = redis.Redis(host=REDIS_HOST, port=REDIS_PORT, socket_connect_timeout=2)
def get_hit_count(client=None, retries=5):
    """Atomically increment and return the 'hits' counter.

    Retries a few times so that the app survives a restart of the
    Redis container (stale pooled connections / DNS not yet updated).
    """
    client = client or cache
    while True:
        try:
            return client.incr("hits")
        except redis.exceptions.ConnectionError:
            if retries == 0:
                raise
            retries -= 1
            time.sleep(0.5)

def container_id():
    """Inside Docker the hostname is the short container ID."""
    return socket.gethostname()

@app.route("/")
def index():
    try:
        count = get_hit_count()
    except redis.exceptions.ConnectionError:
        return (f"Database unavailable (redis://{REDIS_HOST}:{REDIS_PORT}). "
                f"Je suis le conteneur {container_id()}"), 503
    message = (f"Bonjour ! Cette page a été vue {count} fois. "
               f"Je suis le conteneur {container_id()}")
    return PAGE.format(message=message, host=REDIS_HOST)

@app.route("/health")
def health():
    return {"status": "ok", "container": container_id()}

```

- Lines 7–20: read REDIS_HOST (default db-service) and REDIS_PORT from the environment and create a Redis client.
- get_hit_count(): client.incr("hits") adds 1 and returns the new value. If Redis is restarting it waits 0.5 s and retries up to 5 times – this is what makes the robustness test smooth.
- container_id(): inside Docker, the hostname of a container is its short ID, so socket.gethostname() gives the “Je suis le conteneur [ID]” part.
- index() (route /): builds the message and puts it in a small HTML page; if Redis is unreachable it answers HTTP 503.
- health() (route /health): a simple check used by the tests.

4.3 Dockerfile – line by line

```
# =====
```

```

# Visit-Counter - Dockerfile
# Base image: python:3.12-alpine
# * Alpine Linux is ~7 MB (vs ~75 MB for Debian slim and ~350 MB
# for the full python image) -> smallest image, smallest attack
# surface, fastest push/pull.
# * Our dependencies (Flask, redis-py, gunicorn) are pure Python,
# so no compiler / musl problem: wheels install directly.
# =====
FROM python:3.12-alpine

# Python runtime settings: no .pyc files, unbuffered logs (docker logs)
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_PORT=5000

WORKDIR /app

# Create an unprivileged user (security: never run as root)
RUN addgroup -S app && adduser -S -G app -H app

# ---- Docker cache optimisation ----
# Copy ONLY the requirements first: this layer (and the pip install
# below) is reused as long as requirements.txt does not change, even
# if the source code changes.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Source code changes often -> copied last
COPY --chown=app:app app/ .

# Drop root privileges
USER app

EXPOSE 5000

# Production WSGI server (2 workers) instead of the Flask dev server
CMD ["sh", "-c", "gunicorn --workers 2 --bind 0.0.0.0:${APP_PORT} app:app"]

```

Instruction	Why
FROM python:3.12-alpine	Smallest official Python base (Alpine Linux ≈7 MB).
ENV ...	No .pyc files, logs printed immediately (visible in docker logs).
RUN addgroup/adduser	Creates the unprivileged user app (security).
COPY requirements.txt then RUN pip install	Cache trick: dependencies change rarely, so this slow layer is reused when only the code changes.
COPY app/ .	The code, copied last because it changes often.
USER app	From here on, the container does not run as root.
EXPOSE 5000	Documents the port the app listens on.
CMD gunicorn ...	Default start command: a production web server with 2 workers.

RUN vs CMD in one sentence

RUN executes *during the build* and saves the result in the image; CMD is executed *each time a container starts*.

.dockerignore lists files that must not be copied into the image (tests/, .env, caches, .git...): smaller, safer build context.

4.4 Manual deployment (Part 2)

```

docker build -t counter-app:1.0 . # 1. build the image
docker network create app-network # 2. private network
docker volume create redis-data # 3. persistent storage

```

```

docker run -d --name db-service --network app-network \
    -v redis-data:/data redis:7-alpine redis-server --appendonly yes
docker run -d --name web-app --network app-network -p 80:5000 counter-app:1.0
# open http://localhost -> "Bonjour ! Cette page a été vue 1 fois ..."
docker rm -f db-service # robustness test: kill Redis
docker run -d --name db-service --network app-network \
    -v redis-data:/data redis:7-alpine redis-server --appendonly yes
# refresh: the counter continues (data was in the volume)

```



Figure 1: Real output of the robustness test: 8 visits, Redis deleted and recreated, next visit is 9.

4.5 docker-compose.yml and .env (Part 3)

```

name: visit-counter

services:
  web:
    build: .
    image: ${IMAGE_NAME}:${IMAGE_TAG}
    # no container_name: required to be able to scale the service
    environment:
      REDIS_HOST: ${REDIS_HOST}
      REDIS_PORT: ${REDIS_PORT}
      APP_PORT: ${APP_PORT}
    ports:
      # a port RANGE so that each replica gets its own host port
      - "${WEB_HOST_PORTS}:${APP_PORT}"
    networks:
      - app-network
    depends_on:
      - db-service
    restart: unless-stopped

  db-service:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"] # AOF persistence
    volumes:
      - redis-data:/data
    networks:
      - app-network
    restart: unless-stopped

# ---- Bonus: load balancer in front of the 3 replicas ----
# started only with: docker compose --profile lb up -d --scale web=3
lb:
  image: nginx:alpine
  profiles: ["lb"]
  ports:
    - "${LB_PORT}:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
  networks:
    - app-network
  depends_on:
    - web

networks:
  app-network:
    name: app-network
    driver: bridge

volumes:
  redis-data:

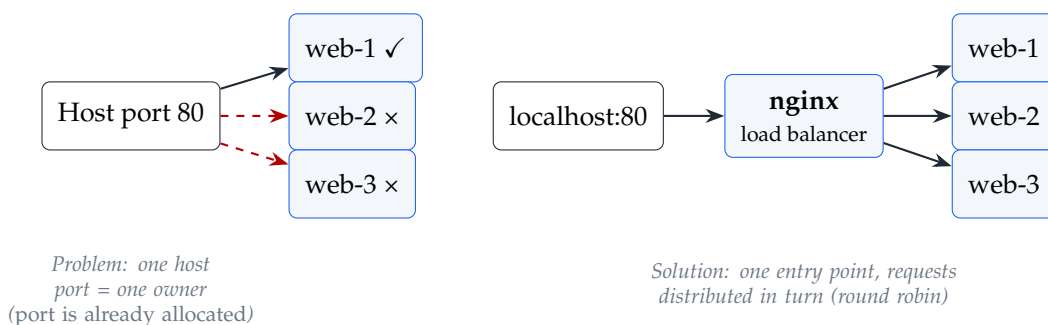
```

```
5  name: redis-data
```

- `${APP_PORT}`, `${WEB_HOST_PORTS}`... are replaced by the values of the `.env` file sitting next to the YAML: no port is written “in hard”.
- `web` has no `container_name`: Docker must be free to create `web-1`, `web-2`, `web-3`.
- `depends_on`: Redis is started before the `web` service.
- `restart`: `unless-stopped`: containers restart automatically (e.g. after a reboot).
- The `lb` service has profiles: `["lb"]`: it is only started when we add `-profile lb`.

```
1  # Values used by docker-compose.yml (no hard-coded ports in the YAML)
  IMAGE_NAME=counter-app
  IMAGE_TAG=1.0
  APP_PORT=5000
  # Host port range for the scaled "web" service (3 replicas)
  WEB_HOST_PORTS=8081-8083
  # Host port of the load balancer (bonus)
  LB_PORT=80
  REDIS_HOST=db-service
  REDIS_PORT=6379
```

4.6 Scaling: why 3 replicas cannot share port 80



A TCP port of the Windows host can only be opened by one program. The first replica takes port 80, the two others fail. Two solutions were implemented: (1) a **port range** 8081–8083 (one port per replica, from `.env`); (2) a **load balancer**: `nginx` alone owns port 80 and forwards each request to one of the replicas through the internal network.

```
1  # Bonus - round-robin load balancer in front of the "web" replicas.
2  # At start-up Docker's embedded DNS resolves "web" to the IPs of ALL
3  # replicas; nginx then distributes requests across them (round robin).
4  upstream visit_counter {
5      zone visit_counter 64k; # shared state -> round robin across all nginx workers
6      server web:5000;
7  }
8
9  server {
10     listen 80;
11     location / {
12         proxy_pass http://visit_counter;
13         proxy_set_header Host $host;
14     }
15 }
```

`server web:5000` is enough because Docker's DNS returns the IP addresses of *all* replicas of the service `web`; `zone` shares the round-robin counter between `nginx` processes so that requests really alternate.

```

NAME                STATUS    PORTS
visit-counter-db-service-1 Up 5 hours 6379/tcp
visit-counter-lb-1    Up 20 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp
visit-counter-web-1    Up 5 hours 0.0.0.0:8082->5000/tcp, [::]:8082->5000/tcp
visit-counter-web-2    Up 26 seconds 0.0.0.0:8083->5000/tcp, [::]:8083->5000/tcp
visit-counter-web-3    Up 35 seconds 0.0.0.0:8081->5000/tcp, [::]:8081->5000/tcp

## 6 requests on the single entry point localhost:80 (nginx)
<- request 1
<- request 2
<- request 3
<- request 4
<- request 5
<- request 6

```

Figure 2: Real output: 6 requests on localhost : 80 answered in turn by the 3 containers.

4.7 Publication and cleaning (Part 4)

```

docker tag counter-app:1.0 oussemabouchaala/counter-app:1.0 # name = user/repo:tag
docker push oussemabouchaala/counter-app:1.0
docker compose --profile lb down --rmi all --volumes --remove-orphans # deep clean

```

Why not `docker system prune -a -volumes`?

That command cleans the *whole* computer: it would also have deleted the volumes of other projects on the laptop (Odo, PostgreSQL). The Compose command only removes what this project created: containers, network, images and volume.

4.8 tests/test_app.py – unit tests

```

"""Unit tests for the Visit-Counter app (no real Redis needed)."""
import sys
from pathlib import Path

import fakeredis
import pytest

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "app"))
import app as visit_app # noqa: E402

@pytest.fixture
def client(monkeypatch):
    fake = fakeredis.FakeRedis()
    monkeypatch.setattr(visit_app, "cache", fake)
    visit_app.app.config["TESTING"] = True
    with visit_app.app.test_client() as c:
        yield c

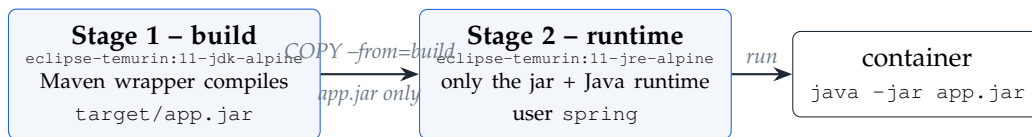
def test_counter_increments(client):
    first = client.get("/").get_data(as_text=True)
    second = client.get("/").get_data(as_text=True)
    assert "vue 1 fois" in first
    assert "vue 2 fois" in second

def test_message_contains_container_id(client, monkeypatch):
    monkeypatch.setattr(visit_app, "container_id", lambda: "abc123")
    body = client.get("/").get_data(as_text=True)

```

fakeredis is an in-memory imitation of Redis, so the tests run anywhere (even in GitHub's servers) without a real database. The 4 tests check: the counter increments (1 then 2), the message contains the container ID, /health returns ok, and the app answers 503 when Redis is down.

5 Part A explained (the tutorial)



- **application-one:** a Spring Boot REST API, GET /api/v1/welcome. Run with `docker run -p 8090:8080 application-one-img:v1`.
- **item-service-mongodb:** a CRUD API on /api/v1/items that stores items in MongoDB.
- In the course, the jar was built on the PC and copied with `FROM openjdk:11 / ADD target/app.jar`. We used a **multi-stage build** instead: no Java or Maven needed on the PC, the deprecated `openjdk` image is avoided, and the final image contains no build tools.

Listing 1: partA-tutorial/item-service-mongodb/docker-compose.yml

```

# Part A - Tutorial: run MongoDB + the Spring Boot item service together
services:
  mongodb-container-one:
    image: mongo:7
    container_name: mongodb-container-one
    ports:
      - "28000:27017" # host 28000 -> container 27017 (as in the course)
    volumes:
      - mongo-data:/data/db

  springboot-with-mongodb-container-one:
    build: .
    image: springboot-with-mongodb:v1
    container_name: springboot-with-mongodb-container-one
    ports:
      - "8090:8081" # host 8090 -> app 8081
    environment:
      # Override application-local.properties (host.docker.internal:28000):
      # inside the compose network the DB is reachable by its service name.
      SPRING_DATA_MONGODB_HOST: mongodb-container-one
      SPRING_DATA_MONGODB_PORT: "27017"
    depends_on:
      - mongodb-container-one

volumes:
  mongo-data:

```

The application finds MongoDB by the service name `mongodb-container-one`; the environment variables `SPRING_DATA_MONGODB_HOST/PORT` override the values written in `application-local.properties` (which pointed to `host.docker.internal:28000`).

```

### POST /api/v1/items (create 2 items)
201 Created -> http://localhost:8090/api/v1/items/6ac4821f3deeb43fd7da200f
201 Created -> http://localhost:8090/api/v1/items/6ac482233deeb43fd7da2010

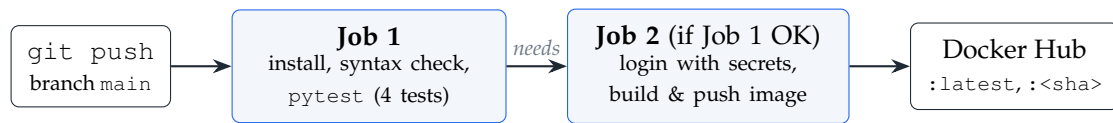
### GET /api/v1/items
[{"_id":"6ac4821f3deeb43fd7da200f","name":"Laptop","description":"Dell XPS","price":1500.0,"categoryId":"c1"}, {"_id":"6ac482233deeb43fd7da2010","name":"Mouse","description":"Wireless","price":25.5,"categoryId":"c2"}]

### the data is really in MongoDB (docker exec + mongosh)
{
  {
    _id: ObjectId('6ac4821f3deeb43fd7da200f'),
    name: 'Laptop',
    price: 1500
  },
  {
    _id: ObjectId('6ac482233deeb43fd7da2010'),
    name: 'Mouse',
    price: 25.5
  }
}

```

Figure 3: Real test: two items created with POST (201), listed with GET, and found inside MongoDB with `mongosh`.

6 Part C explained (the CI/CD pipeline)



```

name: CI-CD Visit-Counter

on:
  push:
    branches: [ "main" ]
    workflow_dispatch: # manual run button (optional)

env:
  APP_DIR: partB-visit-counter
  IMAGE_NAME: counter-app

jobs:
  build-and-test:
    name: 1 - Build & unit tests
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ${ env.APP_DIR }
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: ${ env.APP_DIR }/requirements-dev.txt

      - name: Install dependencies (build)
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt

      - name: Syntax check
        run: python -m py_compile app/app.py

      - name: Run unit tests
        run: pytest -v --junitxml=test-results.xml

      - name: Upload test report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: ${ env.APP_DIR }/test-results.xml

  docker-build-push:
    name: 2 - Docker build & push to Docker Hub
    needs: build-and-test # only if the tests passed
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
  
```

```

6 - name: Build and push image
6 uses: docker/build-push-action@v6
6 with:
6   context: ${ env.APP_DIR }
6   push: true
7   tags: |
7     ${ secrets.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:latest
7     ${ secrets.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:${ github.sha }
7   cache-from: type=gha
7   cache-to: type=gha,mode=max

```

- `on: push: branches: ["main"]`: GitHub starts the workflow automatically on every commit pushed to main.
- `runs-on: ubuntu-latest`: GitHub provides a fresh Linux machine for each job (nothing to install, unlike Jenkins).
- `needs: build-and-test`: the image is published *only* if the tests passed.
- `secrets.DOCKERHUB_USERNAME / DOCKERHUB_TOKEN`: encrypted values stored in the repository settings – the password is never written in the code.
- Two tags: `latest` (newest version) and the commit SHA (to know exactly which code an image contains, and to roll back).

What happened in our runs

Run #1 failed at *Log in to Docker Hub* because the secrets did not exist yet (tests had passed). After adding them, runs #2 and #3 were fully green and the images appeared on Docker Hub.

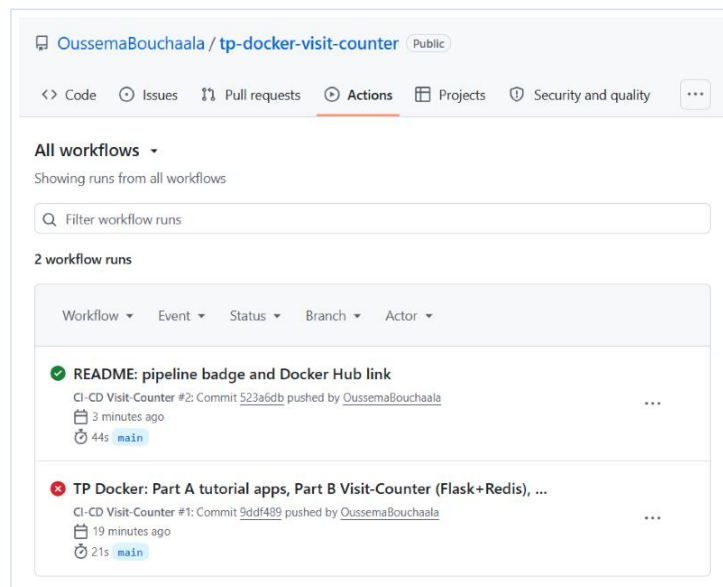


Figure 4: The pipeline runs on GitHub Actions.

7 How to run everything again (cheat sheet)

Open a PowerShell (or the Docker Desktop terminal) in the project folder.

Part B

```
cd partB-visit-counter
docker build -t counter-app:1.0 .
docker compose up -d --scale web=3 # replicas on 8081, 8082, 8083
docker compose --profile lb up -d --scale web=3 # + load balancer on http://localhost
docker compose ps # what is running
docker compose logs -f # live logs (Ctrl+C to quit)
docker exec -it visit-counter-web-1 sh # enter a container
docker compose --profile lb down --rmi all --volumes --remove-orphans # clean
```

Part A

```
cd partA-tutorial\application-one
docker build -t application-one-img:v1 .
docker run -d -p 8090:8080 --name appl application-one-img:v1
curl http://localhost:8090/api/v1/welcome
cd ../item-service-mongodb
docker compose up -d # MongoDB + item-service
curl http://localhost:8090/api/v1/items
docker compose down --volumes
```

Unit tests without Docker

```
cd partB-visit-counter
pip install -r requirements-dev.txt
pytest -v
```

Part C

Edit any file, commit, then **Push origin** in GitHub Desktop: the pipeline starts by itself (tab *Actions* of the repository).

Practical problems we met (and the fix)

- **Slow Internet:** downloading the Maven, JDK and MongoDB images took ≈ 20 min. Pull images once, then everything comes from the cache.
- **Docker Desktop terminal froze** on the animated progress bars of Compose. Fix: add `-ansi never --progress plain`.
- **nginx always chose the same replica:** each nginx worker had its own round-robin counter; fixed with `zone visit_counter 64k;`.
- **Garbled accents** in PowerShell (Ã© instead of é): `[Console]::OutputEncoding = [Text.Encoding]::UTF8`.

8 Glossary

Alpine	Very small Linux distribution used as a base image.
Bridge network	Default type of Docker network on one machine; user-defined bridges add DNS by name.
Build context	The files sent to Docker by <code>docker build .</code> (filtered by <code>.dockerignore</code>).
CI/CD	Continuous Integration / Continuous Delivery: automatic test and delivery on each change.
Gunicorn	Production web server for Python applications.
Layer	One step of an image; layers are cached and shared between images.
Load balancer	Single entry point that spreads requests across several replicas.
Multi-stage build	Dockerfile with several <code>FROM</code> : build in one stage, copy only the result into a small final image.
Redis	In-memory key-value database; <code>INCR</code> increments a number atomically.
Replica	One of several identical containers of the same service.
Secret	Encrypted value stored by GitHub (tokens, passwords) usable by the pipeline.
Volume	Docker-managed storage that outlives containers.
